zühlke
empowering ideas

Accessibility cheatsheet

Practical approaches to universal design, for making your website or web app accessible to everyone

DESIGN Design for accessibility

Use semantic HTML

Semantic HTML helps screen-reader users understand the content's structure and meaning. Ask yourself "What is it?" when writing HTML. Use a logical heading order and structural elements such as `header`, `footer`, `ul`, `p`.

Provide text for non-text content

Content that isn't text is invisible to screen readers. Provide concise descriptions in `alt` attributes for all content images and charts. CAPTCHAs need to be accessible too.

Add labels to form elements

Form inputs need linked `<label>`s for screen-reader access: `<label for="search">Search</label> <input id="search">`. The `placeholder` attribute is not an alternative.

Show where your :focus is

Keyboard users need the currently focused element to be visibly highlighted. Interactive elements should receive focus in a useful order and be focusable by default — use a `<button>`, not a `<span>`.

Use contrasting colours

Text needs enough contrast against its background to be readable.

✓ Readable text ✕ Hard to read

Understandable without colour

Colour shouldn't be needed to understand content or its state. Don't show errors with red text alone — add an icon and a message.

Postcode ⚠ Enter a valid postcode

Write descriptive links

Link text should make sense out of context.

✓ Assignment instructions ✕ Click here

Caption video and audio

Provide text alternatives for audio and video. Captions should be available for pre-recorded and live content. Speech-recognition software can help create captions.

Keep pinch and zoom alive

Pinch-to-zoom helps older and low-vision users. Use `<meta name="viewport" content="width=device-width, initial-scale=1">`. Never set `user-scalable=no` or `maximum-scale=1`.

Use WAI-ARIA only if necessary

Dynamic sites such as single-page apps get complicated for accessibility. Always consider semantic HTML before reaching for WAI-ARIA.

TEST Test for accessibility

1	**Zoom to 400%**	Use browser or document-viewer zoom. Does all text remain visible?
2	**Check keyboard navigation**	Navigate using only arrow keys, Tab, Shift+Tab, Esc, Space and Enter.
3	**Turn your monitor off and use a screen reader**	For example VoiceOver on Apple devices or NVDA on Windows.
4	**Run a Lighthouse accessibility audit**	Open the Lighthouse panel in Chrome DevTools.
5	**Deactivate CSS**	Does the document structure still make sense?

zühlke
empowering ideas

Accessibility cheatsheet

WCAG 2.2 criteria and modern practice worth adding to your checklist

DESIGN Design for accessibility

Make targets big enough

WCAG 2.2 asks for pointer targets of at least 24×24 CSS px, or enough space around them. Aim for 44×44 px on touch.

Keep focus in view

Sticky headers, cookie banners and chat widgets must not cover the focused element. Use `scroll-padding-top` to keep it clear.

Offer an alternative to dragging

Sliders, sortable lists and maps need a single-pointer option such as buttons or a text field.

Don’t make login a memory test

Allow paste and password managers, and support passkeys. No puzzles or transcription without an alternative.

Don’t ask twice, keep help in one place

Auto-fill or offer to reuse details already entered in the same flow. Put help links in the same spot on every page.

Respect motion preferences

Honour `prefers-reduced-motion`. Avoid autoplaying animation, parallax and large motion effects.

Support forced colours and dark mode

Test `forced-colors` (Windows contrast themes) and `prefers-color-scheme`. Don’t convey meaning with background images or shadows alone.

Reach for modern native HTML

`<dialog>` with `showModal()` manages focus for you; `inert` blocks the background; `<details>` gives an accessible disclosure for free.

Name icon-only buttons

Give them an `aria-label` or visually hidden text, and hide decorative SVGs with `aria-hidden="true"`.

Reflow and text spacing

Content should work at 320 px wide without horizontal scrolling, and survive increased line, word and letter spacing.

Check AI-generated alt text and captions

Treat them as a first draft. A person must check accuracy, context and names.

Skip accessibility overlays

Widgets bolted onto a site don’t fix the underlying code and can get in the way of assistive tech.

TEST Test for accessibility

6	**Automate the basics in CI**	Run axe-core (e.g. with Playwright) on every build. It catches common issues; manual testing is still essential.
7	**Emulate user preferences**	DevTools → Rendering: reduced motion, forced colours, dark mode and vision deficiencies.
8	**Test with mobile screen readers**	VoiceOver on iOS and TalkBack on Android — gestures reveal different issues.
9	**Inspect the accessibility tree**	Check each control’s name, role and state in the DevTools Accessibility pane.

Know the rules. WCAG 2.2 (October 2023) is the current W3C standard. The European Accessibility Act has applied since 28 June 2025 to many digital products and services sold in the EU.